

Part B Introduction

Module 3 — Onboarding & Industrialize the Codebase

Instructor **Angelo Sotgiu** · with **Battista Biggio**

Industrial Software Development M.Sc. in Computer Engineering, Cybersecurity and Artificial Intelligence University of Cagliari, Italy

Today:

- what Part B of the course covers, and how it is taught
- the domain of our example codebase: phishing emails
- development environment setup (Linux, or WSL on Windows)
- a guided tour of the example codebase
- your turn: explore the code and find what is wrong with it

What Part B covers (tentative)

Part B is about the *how* of industrial software: the practices and tools that make code reliable, maintainable and safe to change, in a team.

Module	Topic
3	Code quality, refactoring, modularity, environments, packaging, quality tools
4	Testing in practice
5	CI/CD, containers, configuration and secrets
6	Software architecture and APIs
7	AI-assisted development: coding assistants and agents
8	Engineering AI-powered systems: LLM APIs, RAG, agents
9	Security and responsible engineering

How this part is taught: one codebase

All of Part B works on **one codebase**, `heron`, from the first lecture to the last.

- `heron` starts as a messy script that "works on the author's laptop". During the course it becomes a tested, packaged, containerised, AI-powered and secured service.
- Every concept is introduced **together with a real change to heron**: first we see the problem in the code, then the concept that explains it, then we apply the changes that fix or improve it.

Warning: no AI coding assistants, for now

Please **do not use AI coding assistants** (ChatGPT, Copilot, Claude, Gemini, ...) for this course **until Module 7**.

- Only allowed for helping in environment setup.
- In the first modules you train your own eye: finding the problems in the code yourself *is* the skill. An assistant that lists them for you skips the learning.
- The same holds for every lab until Module 7: write and debug the code yourself, so that you understand each tool we add.
- Module 7 is entirely about AI coding tools: how to give them context, how to delegate a task, and how to check what they produce.

The domain: phishing

Phishing is an email that pretends to come from someone you trust (a bank, a colleague, a well-known brand) to make you click a link, open an attachment, or give away a password.

- It is the most common way attackers get into companies: one click from one employee can be enough.
- Given a suspicious email reported by users, deciding whether it is harmless or an attack is called **triage**.
- Triage by hand is slow and repetitive. A tool that reads each email and says *benign*, *suspicious* or *phishing*, with the reasons, saves the analysts a lot of time.

`heron` is that tool

How you recognise a phishing email

A phishing email usually leaves **indicators**: clues that something is wrong.

- **sender impersonation**: the name says "PayPal Support", but the address is on another domain
- **look-alike domains**: `paypal-secure.net` (digit 1 instead of letter l), `amaz0n-rewards.net`
- **deceptive links**: the text shows `www.paypal.com`, but the link goes to a bare IP address
- **failed authentication**: the sender's domain did not allow this email (SPF, DKIM and DMARC: next slide)
- **pressure wording**: "urgent", "suspended", "verify immediately"
- **Reply-To mismatch**: your reply goes to a different domain than the sender's

Each indicator alone proves little. Several together make phishing likely.

Did the sender's domain allow this email?

Anyone can write any address in the `From:` line. Three checks let the **receiving** mail server test that claim. Each domain publishes its rules in DNS:

- **SPF** (*Sender Policy Framework*): the domain lists the servers that may send its emails. The receiving server checks that the server that delivered the email is on the list.
- **DKIM** (*DomainKeys Identified Mail*): the sending server signs the email with a secret key, and the domain publishes the matching public key. A valid signature shows that the email comes from that domain and was not changed on the way.
- **DMARC** (*Domain-based Message Authentication, Reporting and Conformance*): the domain's policy. It requires the domain in `From:` to match a domain that passed SPF or DKIM, and it says what to do when the checks fail: accept, send to spam, or reject.

The receiving server writes the results in the `Authentication-Results:` header, for example `spf=fail, dkim=none, dmarc=fail`. `heron` does not run these checks itself: it only reads that header.

What an email looks like inside

An email is a text file (`.eml`) made of **headers** (one `Name: value` per line), an empty line, and the **body**. These are the first lines of one of our samples:

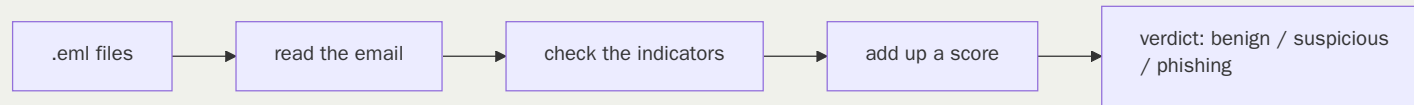
```
Authentication-Results: mx.corp.example; spf=fail smtp.mailfrom=paypal-secure.net; dkim=none; dmarc=fail header.from=paypal-secure.net
From: "PayPal Support" <security@paypal-secure.net>
To: alice@corp.example
Reply-To: recovery-desk@mail-verify.net
Subject: URGENT: your account will be suspended within 24 hours
Content-Type: text/html; charset="utf-8"

<p><a href="http://192.0.2.44/login">https://www.paypal.com/signin</a></p>
```

Count the indicators: failed SPF and DMARC, a brand name on a look-alike domain, a Reply-To on another domain, pressure words, and a link whose text and target differ.

What heron does

heron reads a folder of .eml files and gives each email a verdict:



All the emails in the course are **synthetic**: invented for the course, with reserved domains and IP addresses (example, 192.0.2.x) that belong to nobody.

Your environment: why Linux

In this course you should work in **Linux**: natively on Linux, in the terminal on macOS (which is very similar), and inside **WSL** on Windows. Why?

- servers, CI machines and containers (Module 5) run Linux: code tested on Linux behaves the same in production
- on native Windows, small differences break things: paths use `\` instead of `/`, files end lines with different characters, and many tools (shell scripts, `make`, some Python packages) assume a Linux system
- the commands in the slides, the labs and the CI are written for a Linux shell (bash): they work as shown only there

WSL: Linux inside Windows

WSL (Windows Subsystem for Linux) runs a real Linux system (you can use Ubuntu) inside Windows, without a separate virtual machine to manage. In a PowerShell opened as administrator:

```
wsl --install -d Ubuntu      # then restart, and choose a Linux user name and password
```

- open the **Ubuntu** terminal: from now on, every command of the course runs there
- keep your projects **inside** the Linux file system (for example `~/projects`), not under `/mnt/c/...`: the Windows disk is much slower from Linux and causes permission problems
- your IDE connects to WSL: VS Code with the *WSL* extension, or PyCharm with a WSL interpreter

On macOS and Linux nothing extra is needed: use the normal terminal.

The tools to install

Install these three tools and check their versions:

```
python3 --version      # any Python 3 is enough for today, >=3.12 preferred
git --version
uv --version           # installs Python packages and environments (Module 3, S13)
```

- `git` and Python: from your system's package manager (`sudo apt install git python3` on Ubuntu/WSL)
- `uv`: `curl -LsSf https://astral.sh/uv/install.sh | sh` (official installer, see docs.astral.sh/uv)
- **an IDE**: we suggest **PyCharm** (free Professional licence for students) or **VS Code** with the Python extension
- **a GitHub account** (free, at github.com): you need it today to fork the course repository

Python 3.12 with uv

The course uses **Python 3.12**. The Python of your system can be older: on Ubuntu 22.04, `sudo apt install python3` gives Python 3.10, and heron needs 3.11 or newer from Module 3 on. `uv` can install Python 3.12 for you, next to the system Python, without changing it:

```
uv python install 3.12
python3.12 --version # Python 3.12.x
```

- the command `python3.12` now runs Python 3.12; the system `python3` stays as it was
- if `python3 --version` shows a version older than 3.12, type `python3.12` wherever a slide says `python3`
- from Module 3, heron is a `uv` project: `uv run` then chooses Python 3.12 by itself

Shell basics you will use daily

The terminal is the main interface for everything we do:

```
pwd                # where am I
ls -la             # what is here, including hidden files
cd src             # move into a folder (cd .. goes back up)
cat notes.txt      # print a file
grep -r "TODO" .   # search a text in every file below this folder
```

- `command | grep x` sends the output of one command to another; `command > out.txt` saves it to a file
- the **Tab** key completes names, and the **up arrow** brings back earlier commands: use them, the time savings add up

Git in two minutes

Part A teaches git in detail later. Today you need only three ideas:

- a **repository** is a folder whose whole history git records
- a **commit** is a saved snapshot of all the files, with a message that says what changed; each commit has an ID (a *hash*: a short code of letters and digits that `git log` shows)
- a **branch** is a name for a line of commits; you can switch between branches, and the files in the folder change to that branch's version. The branch name points at the **newest** commit; when you make a new commit on the branch, the name moves forward to it.

The course repository has one branch per lab: you switch to a lab's branch to get its starting code. Today's branch, `lab-01-start`, has a single commit, so it points at that one.

Fork the course repository

A **fork** is your own copy of a repository on GitHub, under your account. You can change your fork as you like; the original repository does not change.

- the course repository is `unica-isde/heron`: you can only read it
- you will write every fix in your forked repository, so each of you needs one
- to fork it: sign in to GitHub, open `https://github.com/unica-isde/heron`, click **Fork**, then **Create fork**; GitHub creates `https://github.com/<your-user>/heron`

You fork only once: from now on, all your work for Part B goes into this fork.

Clone your fork

Clone means: download a copy of a repository, with its whole history, to your computer. You clone **your fork**, not the course repository. Replace `<your-user>` with your GitHub user name:

```
git clone https://github.com/<your-user>/heron.git
cd heron
ls
# notes.txt  results.txt  triage.py  data/
git log --oneline
# 5540b03  stuff
```

`lab-01-start` is the default branch of the repository, so `git clone` already puts you on it. The whole project is one file, `triage.py` (about 80 lines), a `notes.txt`, a `results.txt`, and six sample emails in `data/samples/`. The history has one commit.

Start and solution branches

The course repository gives you **branches**: `lab-01-start` today, then one start branch for each later lab. I will write and push the solution into separate branches like `lab-01-solution`: you can always compare your work with it.

- A new lab can start from a new branch. If your previous work is unfinished or broken, you still start the next lab from working code.

Run it

Try the obvious command:

```
python3 triage.py
```

It crashes. Read the error and `notes.txt`: the script looks for the emails in `/home/lgarcia/mail_export/`, a folder that exists only on the author's computer. Give it the samples folder instead:

```
python3 triage.py data/samples
# checked 6 mails
#   paypal-impersonation.eml -> PHISHING (score 14)
#   receipt.eml -> ok (score 0)
#   ...
```

It prints one verdict and one score for each email (the order of the lines can differ on your machine), a list of "flagged" emails, and it overwrites `results.txt`.

A guided tour of triage.py (1/3): the top

The file starts with a comment, the imports, two variables shared by the whole file, and the list of suspicious words:

```
# quick phishing checker for the mail export - L. Garcia, march 2025
# TODO: make this nicer at some point
import re
import os
import sys

scores = {}
verdicts = []

KEYWORDS = ["urgent", "verify", "suspended", "password", "expires", "act now",
            "congratulations", "winner", "claim", "immediately", "gift card"]
```

`re` is Python's regular-expression module: it searches text for patterns. `os` lists the files of a folder; `sys` reads the command-line arguments.

A guided tour of triage.py (2/3): reading each email

One function does all the work. For each `.eml` file in the folder, it reads the whole file as text, then searches the `From:` and `Subject:` lines with regular expressions:

```
def check_mail(folder="/home/lgarcia/mail_export/", flagged=[]):  
    files = os.listdir(folder)  
    for fn in files:  
        if not fn.endswith(".eml"):  
            continue  
        raw = open(folder + "/" + fn, encoding="utf-8", errors="ignore").read()  
        s = 0  
        try:  
            frm = re.search("From: (.*)", raw).group(1)  
        except:  
            frm = "?"
```

`s` is the score of the current email: every indicator found adds points to it.

A guided tour of triage.py (3/3): scoring and output

Then it adds points: one for each suspicious word, more for bad links, brand names and failed checks. The total decides the verdict:

```
for kw in KEYWORDS:
    if kw in low:
        s = s + 1
...
if s >= 5:
    verdicts.append((fn, "PHISHING", s))
    flagged.append(fn)
elif s >= 3:
    verdicts.append((fn, "suspicious", s))
else:
    verdicts.append((fn, "ok", s))
```

At the end it prints every verdict and writes them to `results.txt`. Read the full file in your editor: it is short.

Your turn: explore

Now explore the code yourselves, possibly in pairs. There is no list to follow: use your experience as programmers.

- run it, in different ways and from different folders
- read every line of `triage.py`; open the sample emails in your editor and compare them with what the script finds
- change something and see what happens
- look at the repository too: its files and its history

Write down everything that looks **wrong, fragile, or hard to understand**, and why. Imagine that from tomorrow you must maintain this code, with a team, for years: what worries you?

Share-out: what did you find?

Takeaways

- Part B teaches industrial practices on one codebase, `heron`: each concept comes with a real fix, and each fix is a commit you can read
- `heron` triages phishing emails: it reads `.eml` files, looks for indicators, and gives a verdict with a score
- environment: Linux (or WSL), the terminal, git, Python 3.12 (installed with `uv`), an IDE, and your own fork of `heron` on GitHub
- at the start, `heron` is one file that works only on its author's machine: you found its problems yourselves
- next: the concepts of code quality, applied one fix at a time